**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**
**Кафедра МО ЭВМ**

# ОТЧЕТ

**по лабораторной работе №4**

**по дисциплине «Алгоритмы и структуры данных»**

**Тема: Бинарные деревья**

**Вариант 9**

Студент гр. 8304 ___ Нам Ё Себ

Преподаватель ___ Фирсов М.А.

Санкт-Петербург

2019

**Цель работы.**

Получить опыт работы бинарными деревьями.

**Постановка задачи.**

Рассматриваются бинарные деревья с элементами типа Elem (в качестве Elem использовать char). Заданы перечисления узлов некоторого дерева b в порядке ЛКП и ЛПК. Требуется:

А) Восстановить дерево b и вывести его изображение

Б) Перечислить узлы дерева в порядке КЛП.

**Описание алгоритма.**

1) Считывание осуществляется при помощи конструкции:

```
while (std::getline(in, currentFileString))
```

2) После считывания данных из переданного файла и сохранения их в массив из данного массива извлекаются очередная пара эл-тов, соответствующих ЛКП и ЛПК записи исходного дерева.

3) Ранее извлеченная пара эл-ов передается в рекурсивную функцию formTree, которая при помощи переданных ей аргументов осуществляет формирование исходного дерева.

4) При помощи функции printTree изображение дерева сохраняется в строчный массив, который, впоследствии, записывается в исходный выходной файл.

**Спецификация программы.**

Программа предназначена для формирования исходного дерева на основании данных, хранящихся в файле F, и записи исходного дерева и его изображения в выходной файл G. Программа написана на языке C++.

**Описание функций и структур данных.**

1) Каждый элемент дерева представляет собой объект класса Node, имеющего 3 поля – поля left и right, являющихся указателями на головы левого и правого поддерева, соответственно, и поле data_, хранящее значение текущего эл-та.

2) Для решения первой поставленной подзадачи была реализована функция printTree, которая осуществляет построчную запись дерева (нарисованного вертикально) во входной массив. Обход эл-ов дерева осуществляется итеративно при помощи контейнера std::stack. Получение строчной записи очередного уровня дерева осуществляется при помощи создания строки, изначально заполненной пробелами, и вставки в данную строку имён узлов на данном уровне по соответствующим им индексам. Пустая узлам дерева соответствуют символы ‘#’.

3) Для решения второй поставленной подзадачи была реализована рекурсивная функция formTree, которая осуществляет формирование исходного дерева при помощи переданных ей ЛКП и ЛПК записей данного дерева. Основная идея реализованного алгоритма заключается в том, что в ЛПК записи дерева имя корня исходного дерева всегда записано последним. Аналогичные рассуждения можно провести, считая левое или правое поддерево исходными, следовательно, имена их корней также находятся в концах, соответствующих им подстрокам исходной записи ЛПК, соответственно, данную функцию можно рекурсивно применить для данных поддеревьев. Выход из рекурсии осуществляется посредством проверки на то, что текущая запись ЛПК пуста, а это означает, что данный узел пуст и очередное поддерево построено.

**Тестирование.**

Таблица 1 – Результаты тестирования программы (изображения дерева представлены в приложении Б)

| **Содержимое файла F** | **Содержимое файла G (после запуска программы)** |
|---|---|
| A B C D E F G H I<br>A C E D B H I G F | NLR: F B A D C E G I H |
| B F G<br>B G F | NLR: F B G |
| F<br>F | NLR: F |
| B E A F C S G<br>E A B S C G F | NLR: F B A E G C S |
| aegnoirngoiq<br>rgmoaimknoamg | Not same trees |
| F G A C D B S d e<br>F G A C D B S d e | NLR: e d S B D C A G F |

**Выводы.**

В ходе работы был получен опыт работы с бинарными деревьями. Исходный код реализованной программы представлен в приложении 1.

## Приложение А. Исходный код программы.

### Main.cpp

```
#include <fstream>
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <algorithm>
#include <stack>

struct Node;
using NodePtr = std::shared_ptr<Node>;


struct Node
{
	Node() = default;

	char name = 0;
	NodePtr left;
	NodePtr right;
};

void readInputData(std::ifstream& in, std::vector<std::string>& data)
{
	std::string tmp;
	while (std::getline(in, tmp))
	{
		if (tmp.back() == '\r')
			tmp.erase(tmp.end() - 1);
		data.push_back(tmp);
	}
}

bool isSameTree(std::string s1, std::string s2)
{
	std::sort(s1.begin(), s1.end());
	std::sort(s2.begin(), s2.end());

	return s1 == s2;
}

bool formTree(std::string const& LNR, std::string const& LRN, NodePtr& head)
{
	if (LRN.empty())
	{
		head = nullptr;
		return true;
	}

	head->name = *(LRN.end() - 1);
```

```
	size_t root_ind = LNR.find(head->name);
	if (root_ind == std::string::npos)
		return false;

	std::string newLeftLNR(LNR.begin(), LNR.begin() + root_ind);
	std::string newRightLNR(LNR.begin() + root_ind + 1, LNR.end());

	std::string newLeftLRN(LRN.begin(), LRN.begin() + newLeftLNR.size());
	std::string newRightLRN(LRN.begin() + newLeftLNR.size(), LRN.end() - 1);

	NodePtr left = std::make_shared<Node>();
	bool leftFormResult = formTree(newLeftLNR, newLeftLRN, left);
	if (leftFormResult == false)
		return false;

	NodePtr right = std::make_shared<Node>();
	bool rightFormResult = formTree(newRightLNR, newRightLRN, right);
	if (rightFormResult == false)
		return false;

	head->left = left;
	head->right = right;

	return true;
}

void printNLR(NodePtr& head, std::ofstream& out)
{
	if (!head)
		return;

	out << head->name << " ";
	printNLR(head->left, out);
	printNLR(head->right, out);
}

void printTree(NodePtr& head, size_t h, std::vector<std::string>& image,
std::ofstream& out)
{
	std::string line(std::pow(2, h + 1) - 1, ' ');

	std::stack<NodePtr> st;
	st.push(head);
	size_t cur_h = 0;

	while (cur_h != h)
	{
		std::vector<NodePtr> nodes;
		size_t step = std::pow(2, h - cur_h + 1);
		size_t ind = std::pow(2, h - cur_h);

		while (!st.empty())
		{
			NodePtr insertElem = st.top();
			st.pop();
```

```
                line[ind] = (insertElem) ? insertElem->name : '#';
                ind += step;

                nodes.push_back(insertElem);
            }
            for (int i = nodes.size() - 1; i >= 0; --i)
            {
                if (nodes[i] == nullptr)
                {
                    st.push(nullptr);
                    st.push(nullptr);
                    continue;
                }
                st.push(nodes[i]->right);
                st.push(nodes[i]->left);
            }
            image.push_back(line);

            std::fill(line.begin(), line.end(), ' ');

            ++cur_h;
        }

}

size_t treeHeight(NodePtr& head, size_t h = 0)
{
    if (!head)
        return h;
    ++h;

    return std::max(treeHeight(head->left, h), treeHeight(head->right, h));
}


int main(int argc, char** argv)
{
    if (argc > 2)
    {
        std::ifstream in(argv[1]);
        if (!in)
        {
            std::cout << "Uncorrect input file\n";
            return 0;
        }
        std::ofstream out(argv[2]);
        if (!out)
        {
            std::cout << "Uncorrect output file\n";
            return 0;
        }

        std::vector<std::string> inputData;
        readInputData(in, inputData);
        if (inputData.size() % 2 != 0)
        {
```

```
                  std::cout << "Input data must consist of pairs (expression and
variable list), entered data is uncorrect\n";
                  return 0;
            }
            size_t ind = 1;
            for (auto it = inputData.begin(); it != inputData.end(); it += 2,
++ind)
            {
                  out << "№" << ind << "\n";
                  // Left (корень) Right
                  auto& LNR = *it;
                  // Left Right (корень)
                  auto& LRN = *(it + 1);

                  LNR.erase(std::remove_if(LNR.begin(), LNR.end(), [](char c)
{return c == ' '; }), LNR.end());
                  LRN.erase(std::remove_if(LRN.begin(), LRN.end(), [](char c)
{return c == ' '; }), LRN.end());

                  bool checkIsSameRes = isSameTree(LNR, LRN);
                  if (checkIsSameRes == false)
                  {
                        out << "Not same trees\n";
                        continue;
                  }

                  NodePtr head = std::make_shared<Node>();
                  bool formResult = formTree(LNR, LRN, head);
                  if (formResult == false)
                  {
                        out << "Uncorrect data\n";
                        continue;
                  }

                  std::vector<std::string> image;
                  printTree(head, treeHeight(head), image, out);
                  for (auto line : image)
                        out << line << "\n";

                  out << "\nNLR: ";
                  printNLR(head, out);
                  out << "\n\n";
            }
      }
}
```

**Приложение Б. Пример изображения деревьев.**







